

Lecture 21:

Google File System (GFS)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison

Today's Topic: GFS

S. Ghemawat, H. Gobioff, and S.-T. Leung. [The Google File System](#).

SOSP, SIGOPS Oper. Syst. Rev., 37(5):29-43, 2003. **SIGOPS Hall of Fame Award**

- Has evolved many iterations since then...

Why Today's Paper?

- Ideas not particularly novel, but...
- Very influential – shows how to build a large-scale distributed fs w/ faults
- Use knowledge of workload and environment (build on Linux)
- Simple at first (centralized master, some soft state)
- Open-source implementation: Hadoop Distributed FS (HDFS)

Today

- Motivation
- Architecture and Scalability
- Recovery and Consistency
- Optimizations and Details

GFS Motivation

Design for specific environment and workload

Computing environment?

-
-

Google workload characteristics?

-
-
-
-

Opportunity for Co-Design

- Do not need general-purpose file system
 - No need to be backwards-compatible with existing applications
 - Does not need to adhere to POSIX specification
- Avoid difficult FS features:
 - Read directory (make new directory structure), links, reading from open, deleted file

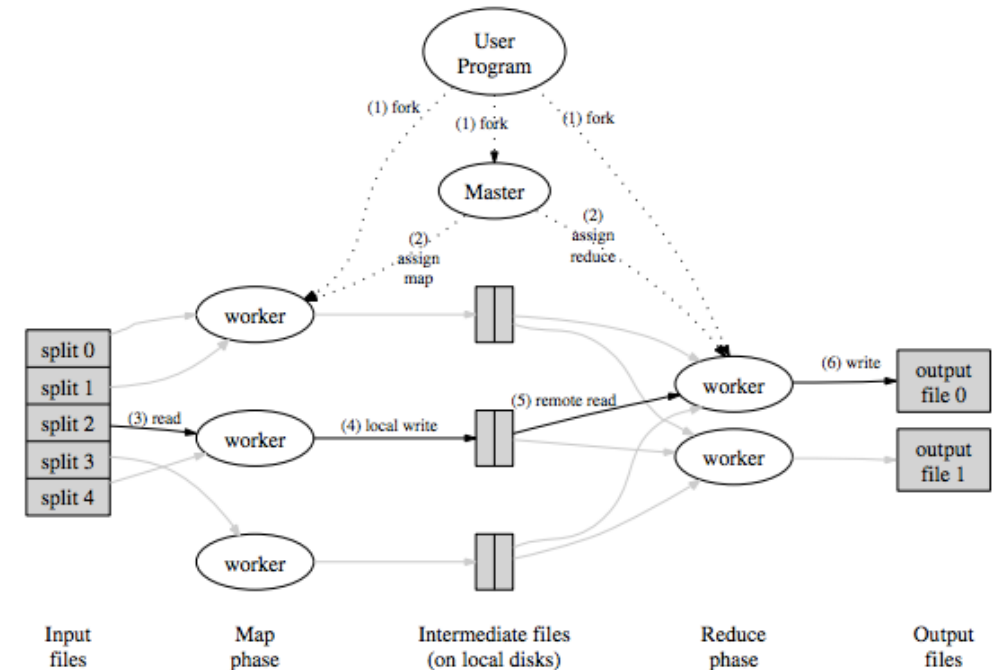
What Workloads?

MapReduce

- Many concurrent mappers and reducers
- Mappers
 - Read entire input file (in chunks)
 - Local compute over data
 - Write to local files (not GFS)
- Reducers
 - Run after all mappers done
 - Read from intermediate files created by mappers
 - Local compute over data
 - Append to separate output files

Producer/consumer

- Many producers **append** work to shared file concurrently
- Consumers read, do work, and **append** to output file



GFS Interface?

Non-POSIX file system interface w/ expected operations
creat, delete, open, read, write, close

Snapshot: create copy of file at low cost

- Copy-on-write (keep new copy on local machine)

Record append: guarantee to be atomic

- Multiple clients concurrently append to one file
- Why introduce record append as own operation vs. write-at-end-of-file?

GFS: Architecture and Scaling

How is the overall system designed?

Why is there a single master?

Why doesn't the single master limit scalability?

What exactly does the master do, and what does it avoid doing?

Why are GFS chunks so large?

What workload assumptions make this architecture work?

Environment: Machines Fail

Commodity components, including Linux; Large scale
Machines storing data may permanently and/or temporarily fail

1) Must **replicate** data

Less structured than RAID (no static computation to get locations)

- Machines come and go
- Capacity of each chunk server may vary
- Different data may have different replication levels (e.g., 3 vs 5 copies)

Problem: How to map logical to physical locations?

2) Must **recover** (respond to machines stopping and starting)

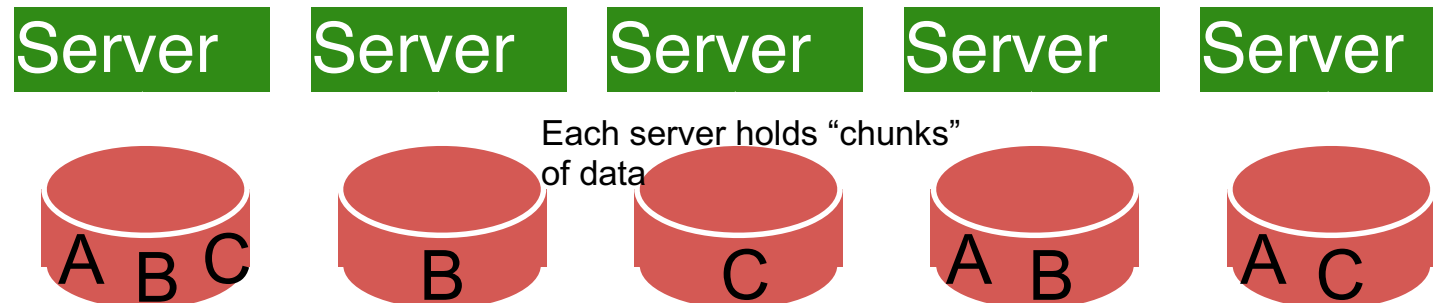
Machine may be dead forever

Must identify and replicate lost data on other servers

Machine may come back; disk space wasted with extra replicas

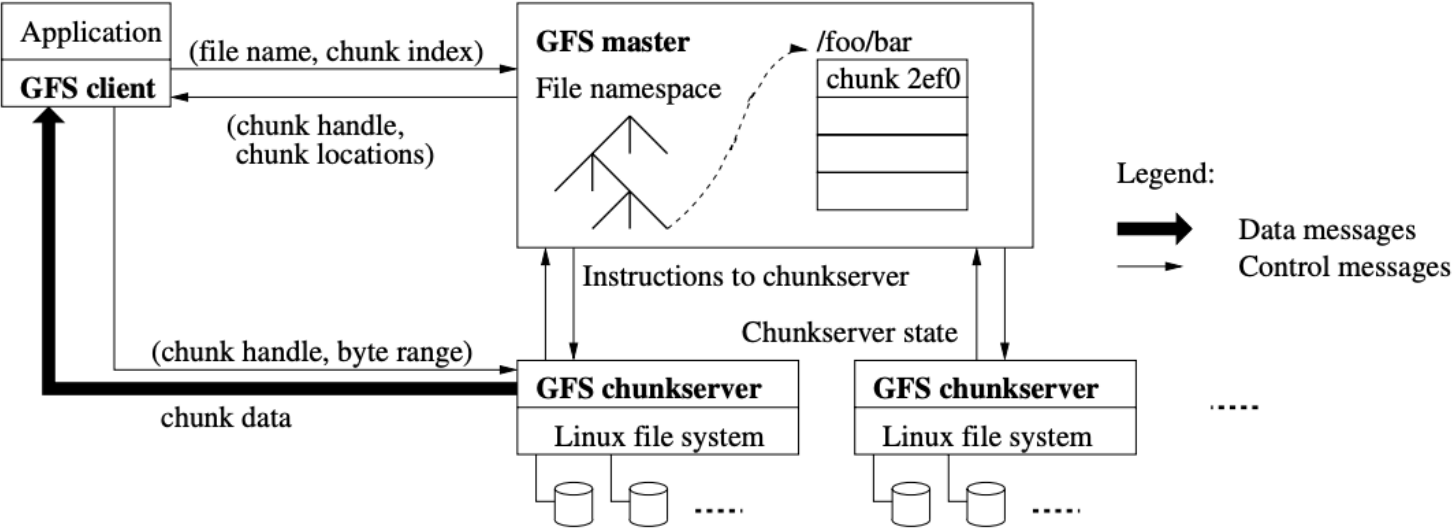
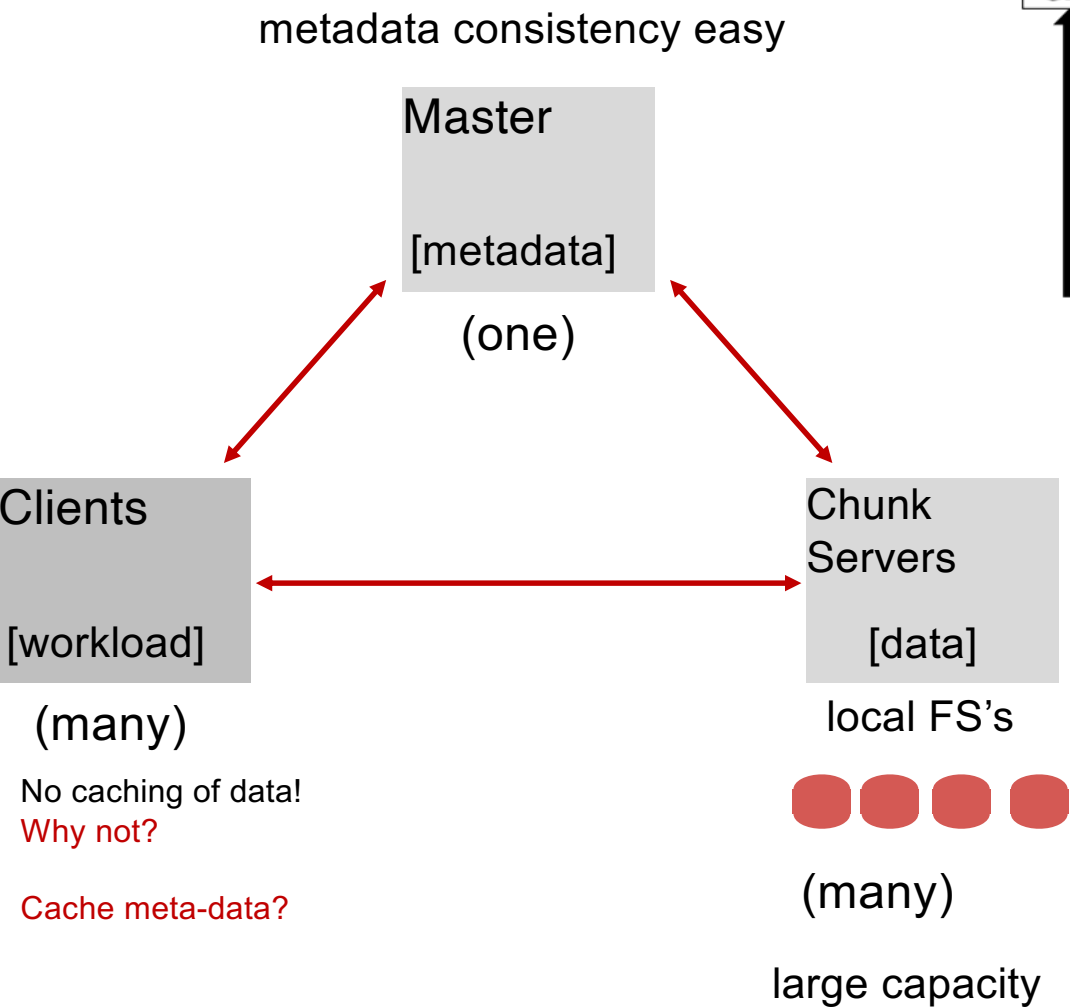
Identify number of replicas and choose to remove extras

Problem: Finding copies of data + maintaining replicas is difficult w/o global view of data



GFS Architecture

Communication between each?



Master Metadata

Master

chunk map:

handle	locations
924	w2,w5,w7
521	w2,w9,w11
...	...

client

Worker w2

Local FS

/chunks/924 => data1

/chunks/521 => data2

...

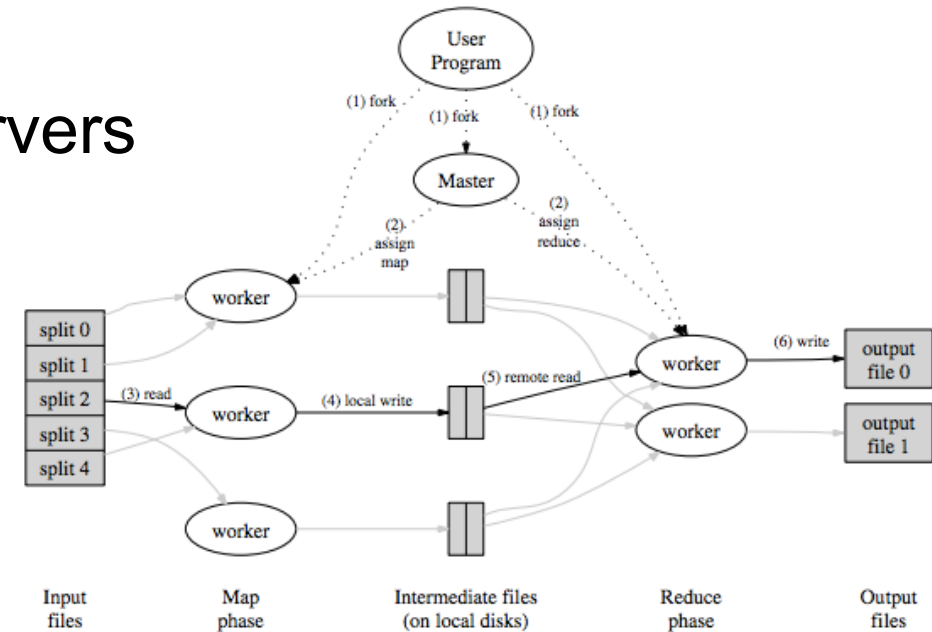
Reads: What can be cached on client?

What is a Chunk?

Break GFS files into large chunks (e.g., 64MB);
unit of replication; chunks not split across chunkservers

Interaction with MapReduce?

-



Chunk servers store physical chunks in Linux files

Master maps logical chunk to physical chunk locations

How to pick Chunk Size?

GFS uses large chunks, e.g., 64MB
How does chunk size affect size of master data structs?

What if Chunk Size Doubles? Any disadvantages to making chunks huge?

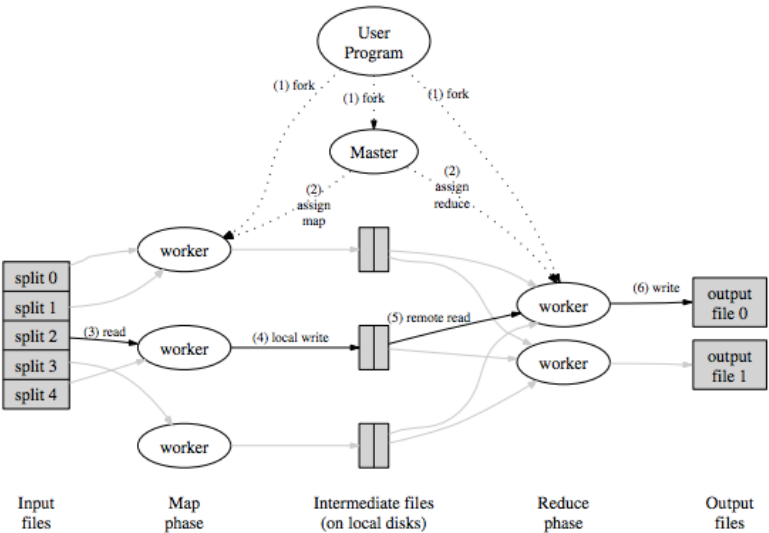
Master

file namespace:
/foo/bar => 924,813
/var/log => 123,999

chunk map:

logical	phys
924	w2,w5,w7
813	w1,w8,w9
...	...

File namespace and chunk map fit 100% in RAM
Advantage?



Recovery and Consistency

What state must survive a master crash?

What state can be reconstructed after a crash, and why?

What kinds of consistency does GFS actually provide?

How does GFS ensure that replicas apply writes in the same order?

Why does GFS add record append as a special operation, and what guarantees does it give?

How to Handle Master Crashing

Two data structures to worry about

1) How to make **namespace** persistent?

Persist to disk before ack client

Where should logs be located?

-
-
-

Result: High availability when master crashes!

General approach: Primary/backup

Master

file namespace:
/foo/bar => 924,813
/var/log => 123,999

chunk map:

logical	
924	
813	
...	...

Master Crash: Chunk Map Consistency

2) How to make chunk map persistent?

-

Approach:

After crash (and when maintaining replicas), master asks chunkserver which chunks each has

What if chunkserver dies too?

-

Master

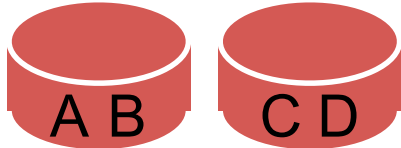
file namespace:
/foo/bar => 924,813
/var/log => 123,999

chunk map:

logical	
924	
813	
...	...

Master

Worker



Consistency: Different depending on use

Consistency of namespace operations?

-

Consistency of data operations?

Terminology

- Consistent
 - All replicas are identical → All clients see same data regardless of which replica they read
- Defined
 - Consistent AND state reflects mutation in entirety (atomic update with no intermixed ops); defined > consistent
- Inconsistent
 - Replicas are not identical (only subset updated)

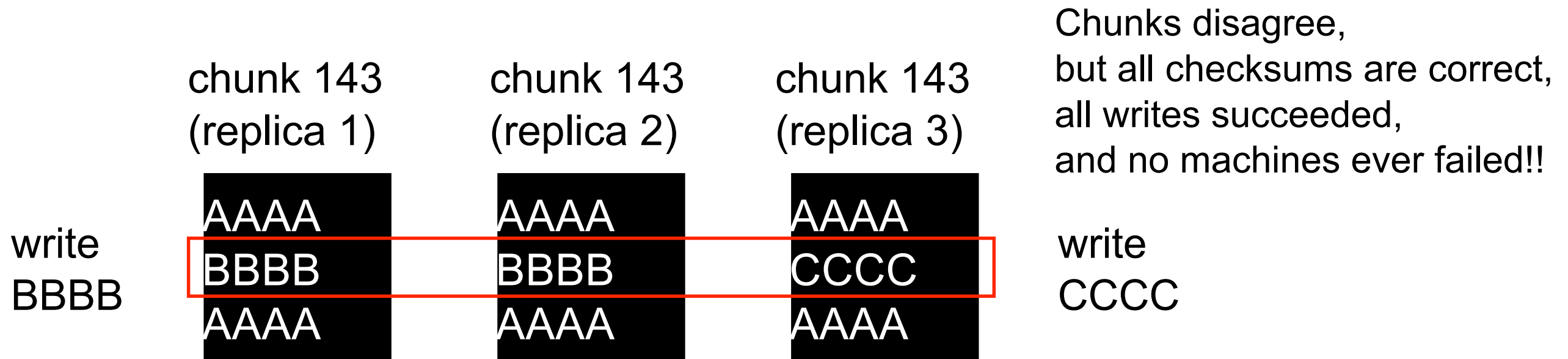
	Write	Record Append
Serial success	<i>defined</i>	<i>defined interspersed with inconsistent</i>
Concurrent successes	<i>consistent but undefined</i>	
Failure	<i>inconsistent</i>	

Chunkserver Consistency

Challenge:

Want physical chunks on different chunkservers to be consistent with one another

What about concurrent writes (or appends) from different clients?



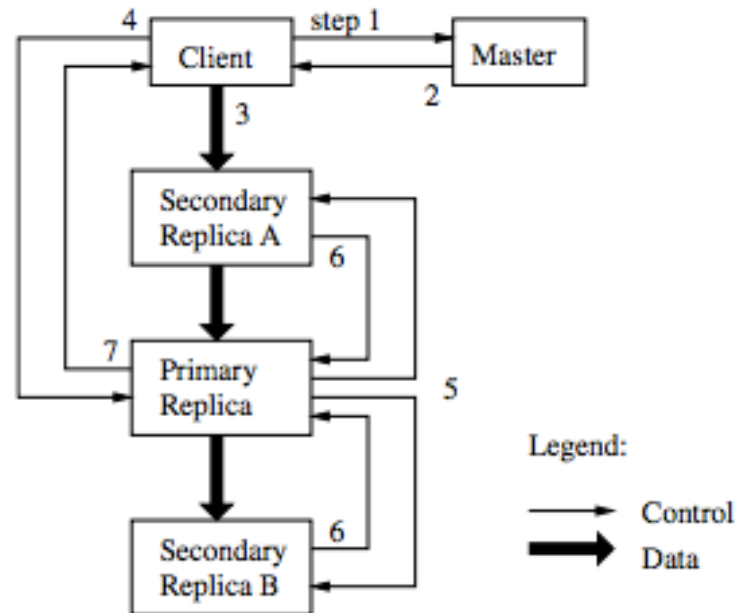
Chunkserver Consistency: Writes

GFS must “serialize” writes across chunkservers

- Decide order of writes and ensure order is followed by every chunkserver

How to decide on order of writes?

-



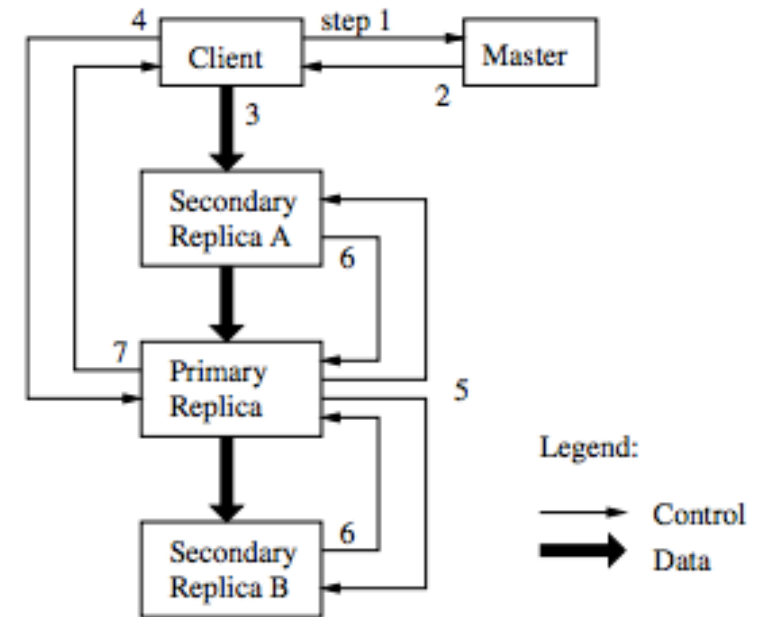
Primary: assign seq num to each write

Performance Optimization:
Correctness:

Primary Replica

Master chooses primary replica for each logical chunk

What if primary dies?



Writes vs. Record Appends

Concurrent writes spanning two chunk

What consistency? Example?

	Write	Record Append
Serial success	<i>defined</i>	<i>defined interspersed with inconsistent</i>
Concurrent successes	<i>consistent but undefined</i>	
Failure	<i>inconsistent</i>	

Record append: GFS chooses offset

- Guarantee: At-least-once atomically appended
- Ensure record fits within chunk, if not pad and tell secondaries to do same, retry at next chunk

If failures before finally returning success, what consistency?

Implication for applications?

Optimizations and Details

How does GFS perform snapshots?

Why does GFS use a flat namespace instead of Unix-style directory structures?

Why is lazy garbage collection better than eager deletion here?

How does GFS detect corruption if replica disagreement is not enough?

File Snapshot

Efficient way to copy a file

Use Copy-On-Write (COW)

How to ensure chunks are not being modified during snapshot?

Steps:

- First,
- Add create(F') to op log, point F' to same chunk
- If a client wants to write, take a copy at that time

Copy on the same chunkserver → disk is faster than network

Flat Namespace

Unix/Linux: Directories are special files

- Inode where data contents are directory entries

GFS: Directly map each pathname separately to metadata

- No per-directory data structures, prefix compression of pathnames

Motivation?

-

On Unix/Linux:

- dir ops are serialized
create /foo/a.txt, create /foo/b.txt

What needs to be protected?

GFS:

→ Allow concurrent creations within a single directory

File Deletion and Garbage Collection

When delete, just rename to hidden name

Immediately log the operation but lazily gc hidden files (after 3 days)

- Remove name of hidden file

Garbage Collection:

- Identify orphan chunks -- those not pointed to by any file (similar to fsck)

Chunk Servers:

- Tell Master which chunks it currently has

Master:

- Inform Chunk Server to drop those chunks

Simpler, more elegant approach than eager delete!

Data Integrity

Stored blocks can become corrupted over time

- Files stored on Linux which has bugs – sometimes silent corruptions
- Files stored on disk, and disks are not fail-stop

Rare events become common at scale

Could one detect errors by comparing across replicas? Why or why not?

-

Chunkservers maintain per-chunk CRCs (64KB)

- Verify CRCs before returning read data

Record appends – incremental cksum, verify on read

Overwrites – read, verify, recompute, and write new cksum

GFS -> Colossus

Problems?

1)

2)

Open-source HDFS has same problems

GFS replaced with a new design, Colossus

Colossus supports much larger file systems – all about Meta-data scalability

GFS Summary

Leverage knowledge of workload and environment

- Don't be completely general (At least at first! Get something working well!)

- Use appropriate interfaces (record append) and weaker consistency for data (strong for metadata)

Handles failures at scale

- Location of chunks soft state

- Simple recovery for back-up of master

- Checksum of all data

Simplicity of centralized master

- Not able to stay this way for long

Built on top of Linux

- Don't reinvent system software

Neat Basic ideas

- Garbage collection of old chunks on chunk servers

- Flat file name space for concurrent operations

- Separate control and data plane for faster updates

- Simple leases for per-chunk primary